MalwareXplore

Project Team

Minam Faisal 21I-1901
Momenah Saif 21I-1909

Session 2021-2025

Supervised by

Mr. Muhammad Abdullah Abid

Department of Cyber Security

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2025

Contents

List of Figures

Chapter 1

Introduction

This project aims to create an automated system for detecting PDF malware. By integrating multiple analysis methods and automation, the system will enhance malware detection accuracy while reducing manual effort. Additionally, the system will incorporate a learning architecture that enables instances to collaborate and continuously improve the detection model. This document is intended for developers, project managers, malware analysts, cybersecurity experts, and testers who are involved in the development, deployment, and evaluation of the system.

1.1 Problem Statement

Malware analysis is facing challenges due to fragmentation, manual dependency, and collaboration issues. Static and dynamic analysis are used independently, leading to incomplete results, and analysts must manually examine logs, investigate suspicious activity, and gather findings from various tools. This process is inefficient and prone to human error. With the increasing number of malware threats, relying solely on human analysis leads to delayed detection. When systems learn new malware detection skills, these are often limited to individual systems, reducing overall detection accuracy and system evolution. The proposed solution addresses these issues by combining static and dynamic analysis into one unified system for more detailed results. AI automation reduces manual work, speeds up detection, and enhances accuracy. Additionally, a decentralized system allows for collaboration by sharing detection techniques without compromising sensitive information. This solution makes malware detection more efficient, accurate, and globally collaborative.

1.2 Scope

The malware detection system is built to assist malware analysts by automating both static and dynamic analysis, specifically targeting PDF files, which are often used to deliver hidden threats. The tool's main goal is to bring both types of analysis together in one platform, making it easier for analysts to identify and examine potential malware without switching between multiple tools. The results will be displayed on a clear, easy-to-use dashboard, with the option to download comprehensive reports for deeper analysis. Using federated learning, the system continuously improves its detection accuracy by learning from data across different systems without sharing sensitive information. This means the tool not only gets smarter over time but also adapts quickly to new and emerging threats. Continuous updates ensure the system stays up to date with the latest malware trends, providing analysts with an efficient, reliable, and evolving solution for malware detection.

1.3 Modules

Here is a breakdown of each module along with its explanation:

1.3.1 Malware Analysis Module

This module automates the analysis of malware in PDFs. It includes web portal where user can easily upload the file for analysis.

- **Feature 1:** Automated Malware Analysis
- **Feature 2:** Web Portal for PDF Upload

1.3.2 Data Management Module

Storing and managing the analysis data will be done by this module. By using both centralized and distributed storage it ensures data security and availability.

- **Feature 1:** Centralized and Distributed Storage
- **Feature 2:** Data Privacy and Security

1.3.3 Visualization and Reporting Module

This module includes the dashboard that display the analysis results through charts and graphs. It also provides report of analysis results which user can download.

- **Feature 1:** Visualization Dashboard
- **Feature 2:** Downloadable Reports

1.3.4 Federated Learning Module

This module focuses on malware detection part. It ensures that AI models are trained on decentralized sources and enhanced detection by continuously improving its detection capabilities and preserving privacy.

- **Feature 1:** Federated Learning Integration
- **Feature 2:** AI-Driven Learning

1.4 User Classes and Characteristics

Following are the users and their characteristics:

User class	Description
Malware Analyst	Malware Analyst use the system to automate static and dynamic analysis of PDF Files. Analysts can initiate scans, examine results on the dashboard, and download detailed reports. Analysts benefit from AI-driven detection and federated learning.
System Administrator	Responsible for managing the system infrastructure. Administrators ensure updates, monitor performance, and manage system security. They oversee access control, system integrations, and privacy settings for sharing models across organizations, ensuring secure collaboration and proper system maintenance.

1.4 User Classes and Characteristics

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of our project.

2.1 Event Response Table

We used Event- response table as requirement gathering technique. Event- response table is for real-time system in which most of the functionalities are performed at backend.

Table 1

Event	Source of Event	Precondition	Response	Post condition
User Create account	User	None	Account created	User required to login
User login attempt	User	Account existence	Authenticate credentials	User logged in or denied
PDF file uploaded	User	User access to web portal and log into account	File is submitted to Flask backend	File is accepted for analysis
Invalid file format uploaded	User	User attempts to upload non-PDF file	Reject file and notify the user of supported formats	File rejected, user notified
Static analysis of PDF	Flask and Python code	PDF file uploaded and analysis type is selected	Analyze file structure and metadata without executing the file	Static analysis results stored in Elasticsearch and sent to Grafana.
Dynamic analysis of PDF initiated	Flask and code of Cuckoo	PDF file uploaded and	Execute PDF in a virtual	Dynamic analysis results stored in Elasticsearch

	sandbox API call	analysis type is selected	Environment of cuckoo sandbox.	and sent to Grafana.
Both type of analysis of PDF initiated	Flask and code of Cuckoo sandbox API call and python code	PDF file uploaded and analysis type is selected	Analyze file structure and metadata also execute PDF in a virtual Environment of cuckoo sandbox.	Static and Dynamic analysis results stored in Elasticsearch and sent to Grafana.
PDF identified as malicious	Flask and Machine learning model	Analysis completed	Show file as malicious and the button for visualization appears.	Malicious file stored in databases.
Dashboard request for analysis visualization	User	Malware analysis completed and stored	Grafana fetches analysis results and visualizes them.	Visualization displayed to user
Request to download report of analysis	User	Malware analysis completed, stored and visualized	Grafana combines the results in a report and give option for download	A report has been downloaded into the users machine.
Local model receives analysis results	Flask (Federated Learning)	Analysis completed	Local model self-trains using results.	Local model updated
Global model update request initiated	Flask (Federated Learning)	Local models have updated	Global model aggregates updates from local models	Global model updated and redistributed

2.2 Functional Requirements

Following are the Functional Requirements of our Project:

2.2.1 Centralized and Decentralized Storage

Following are the requirements:

- **FR1:** The system will store the malware analysis results in both centralized (Elasticsearch) and decentralized (Federated local models) ways.
- **FR2:** The system will make it easier to find and use information from different locations, making it less likely for data to be lost or stolen.

2.2.2 Data Privacy & Security

Following are the requirements for this module:

- **FR3:** All PDF files, analysis results and user information will be encrypted in transit and rest states.
- **FR4:** The system will ensure that the analysis of the PDFs the user have uploaded are only shown to that specific user.

2.2.3 Federated Learning Integration

- **FR5:** The system will enable the local models to share the data of malicious pdf to global model for training without the need to share the malicious pdf or sensitive data.
- **FR6:** The system will regularly update its local model which will be sent to global model and all the other participating nodes will be refined.

2.2.4 AI-Driven Learning

- **FR7:** The system will automatically update its machine learning model on new features for more accurate detection of malwares.
- **FR8:** The system will support continuous model improvement by adding new analysis data into the training dataset.

2.2.5 Web Portal for PDF Upload

- **FR9:** The system will allow user to create accounts.
- **FR10:** The system will allow user to upload PDF files and will reject any other format files.

2.2.6 Automated Malware Detection

- **FR11:** The system shall do static analysis on uploaded files to know its metadata and much more.
- **FR12:** The system shall do dynamic analysis on uploaded files by running them in virtual environment of cuckoo sandbox.

2.2.7 Visualization Dashboard

- **FR13:** The system will allow the user to access the results on analysis on Grafana dashboard in form of charts.
- **FR14:** The system will allow the user to download the analysis results in form of report from Grafana dashboard.

2.3 Non-Functional Requirements

This section outlines the non-functional requirements that define the system's quality attributes such as reliability, usability, performance, and security.

2.3.1 Reliability

The system must provide a high reliability for malware analysis, data management and federated learning operations so that failures are not repeated frequently within the system.

- **RELI-1** If a system failure occurs during the uploading of files or during the analysis, the system will alert the user, then automatically continue from the last known state.
- **RELI-2:** Mechanism for automated failure detection must be in place so that there is detection and notification of failure during malware detection or update in federated learning.

2.3.2 Usability

The usability of the system will ensure a user-friendly experience, especially in case of malware analysts and security personnel, who use a dashboard and reporting tools.

- **USE-1:** The web portal must assure that the interface is user friendly and that can be achieved with obvious visible indicators of upload progress and analysis status as well as reports are available.
- **USE-2:** The dashboard user interface must enable users to select charts and graphs for graphical display of analysis output, to their taste.
- **USE-3:** There must be a help section, on the web portal, enabling the users to guide through the upload and analysis and reporting processes.

2.3.3 Performance

Performance requirements will make the system functional at different times, allowing many user requests and other processes of malware analysis to be performed within a timely manner.

- **PER-1:** At normal load, the system shall be able to complete its static and dynamic analysis on a 10MB PDF quickly.
- **PER-2:** The model updates of the federated learning should be processed and integrated into the system within few minutes of receiving new data from decentralized sources.

2.3.4 Security

The security aspect is what ensures the integrity of the system and the privacy of user information.

- **SEC-1:** All data uploads, downloads, and any communications between the web portal and back-end services should be end-to-end encrypted.
- **SEC-2:** All data of analysis should be encrypted with strong algorithm, including PDF uploads and results.
- **SEC-3:** Backend services and databases used by the system shall be accessible only by authorized personnel through authentication.
- **SEC-4:** Federated learning updates and data exchanges should be performed over a secure environment that does not allow sharing of sensitive data between organizations.

2.4 Domain Model

For the system, this domain model is uploading and analyzing PDFs with a federated learning model. Users are represented with attributes like userID, name, email, password. The PDF analysis produce static and dynamic results, time-stamped with start and end times. The result from analysis is stores with associated data display. The federated learning model can process training data and include PDF files, update periodically.

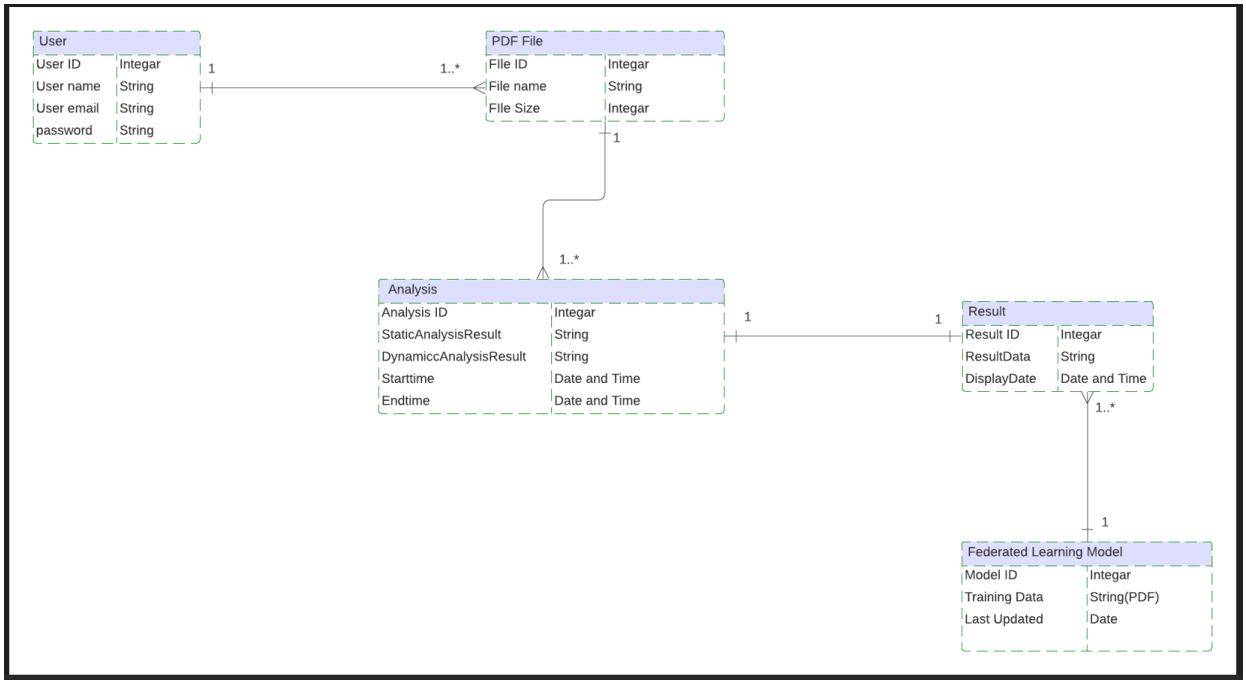

Figure 2.4

Chapter 3

System Overview

The ***PDF Malware Detection System*** automates the analysis of malicious PDF files using both static and dynamic techniques. It allows users to upload files via a web portal, with results securely stored in a centralized and distributed database. The system includes an interactive dashboard for visualization and provides downloadable reports. Additionally, the system integrates a Federated Learning module, enhancing detection capabilities by continuously training AI models on decentralized data.

3.1 Architectural Design

This malware detection architecture is divided into four major modules: the Malware Analysis Module, Data Management Module, Visualization and Reporting Module, and Federated Learning Module. Each module satisfies distinct functionalities; however, the modules work together to achieve seamless analysis and reporting experiences: -

1. Malware Analysis Module

Description:
This module automatically analyses uploaded PDF files for malware threats. The users upload the files through a web interface, and then all those files are statically and dynamically analyzed by the module for the presence of malicious behavior.

Interactions:
Requests the Data Management Module to store and retrieve files uploaded by users as well as the results of analysis performed by the module.

2. Data Management Module

Description:
It makes sure that the uploaded files, the results of analysis, and all related data are stored, organized, and secured. It maintains data integrity based on availability, security, and privacy through a centralized and decentralized storage mechanism.

Interactions:
Received results of analysis and file uploads from the Malware Analysis Module. Serves data appropriate for the visualization reporting module to display results of analysis. It shares data sets with a module for federated learning to back model training as well as enhancement of algorithms used for detection.

3. Visualization and Reporting Module

Description:
This module is supposed to provide the user with an easy user interface to view the results of the malware analysis graphically, say, in charts and graphs. Thus, the detailed reports of the malware analysis may be downloaded thus allowing the ultimate user to have a comprehensive view of the results of the analysis.

Interactions:
Retrieve data from the Data Management Module for the creation of graphical representations of analysis results.

4. Federated Learning Module

Description:
In the Federated Learning Module, machine learning improves malware detection capabilities. Models are trained on decentralized data, such that user privacy is preserved, and detection algorithms constantly improve because of learning from rich historical data from multiple sources.

Interactions:
It works in conjunction with the Malware Analysis Module to draw improvements in its algorithms through learned patterns from past data analyzed. It provides access to relevant datasets from the Data Management Module for training and fine-tuning machine learning models.

Initial Design:

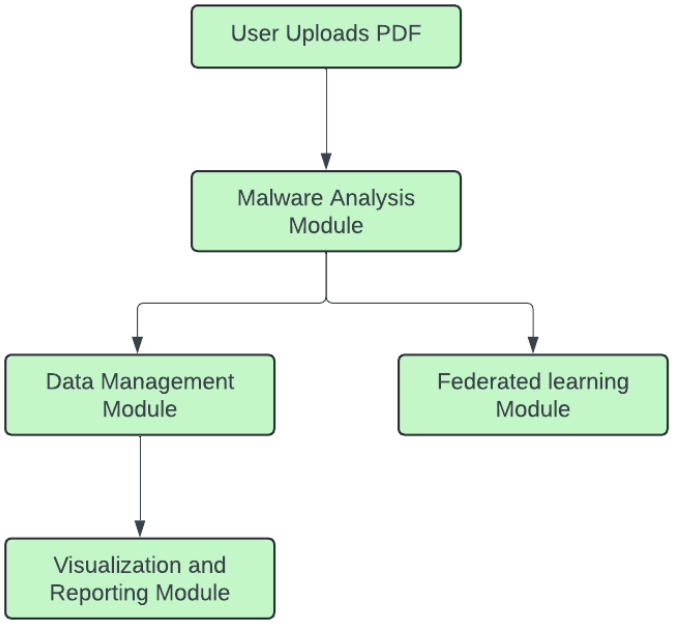

Figure 3.1.1

Final Architecture:

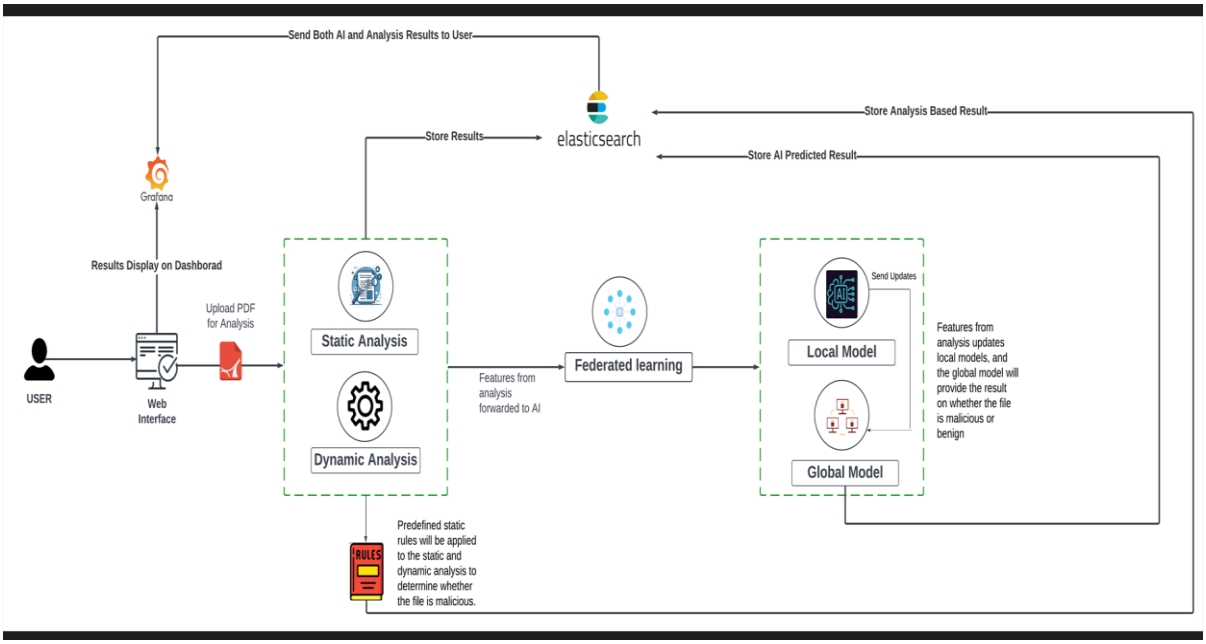

Figure3.1.2

3.2 Design Models

The applicable models for our project using ***procedural approach*** are:

Activity Diagrams:

Activity Diagrams are based on events we found in event response table:

Login/Sign up

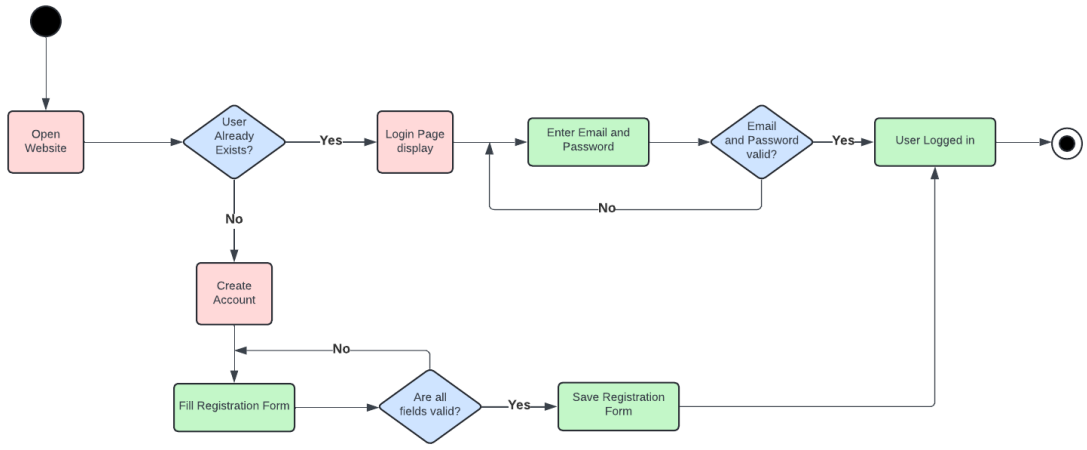

Figure 3.2.1

Edit Profile

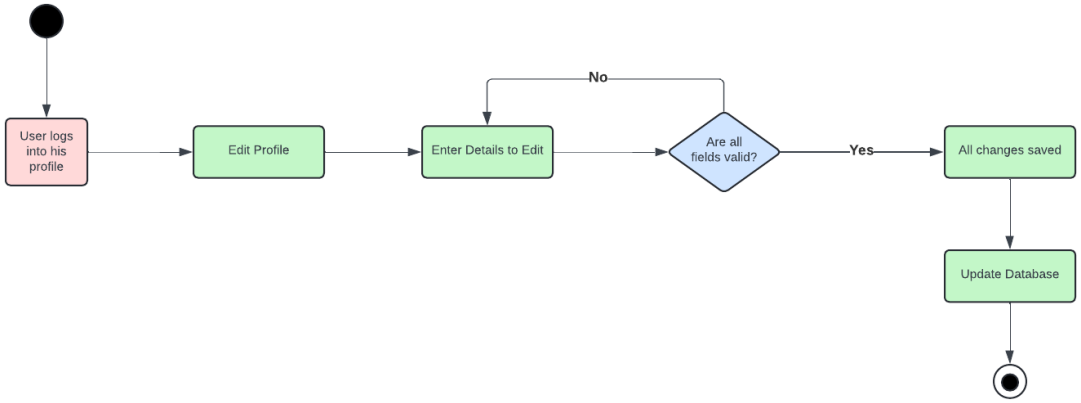

Figure 3.2.2

Upload PDF

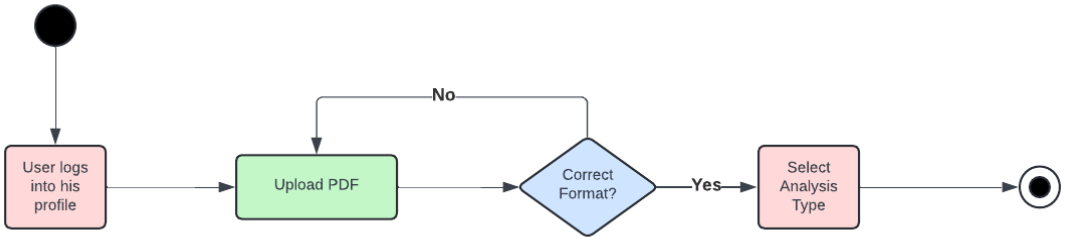

Figure 3.2.3

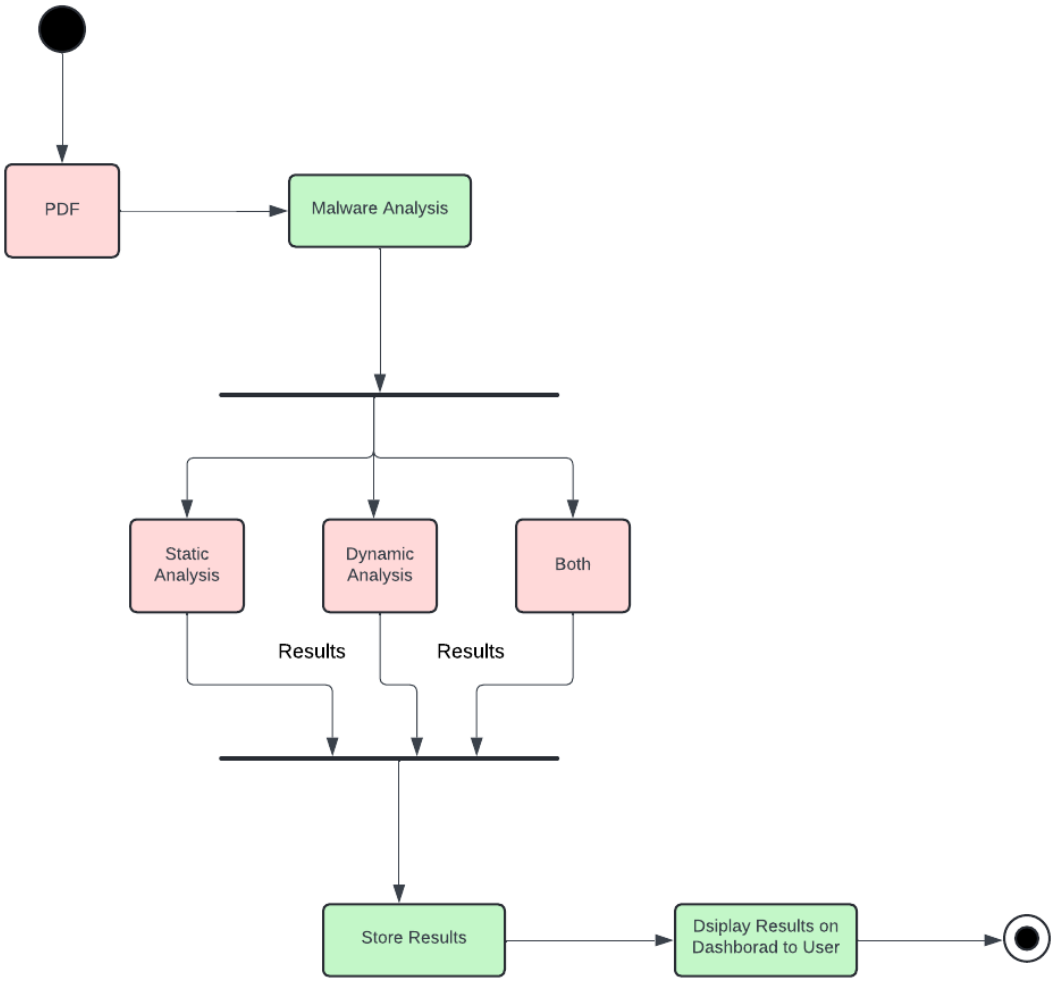
PDF
Malware Analysis
Static Analysis
Dynamic Analysis
Both
Results
Results
Store Results
Dsiplay Results on Dashborad to User

Figure 3.2.4

Federated Learning

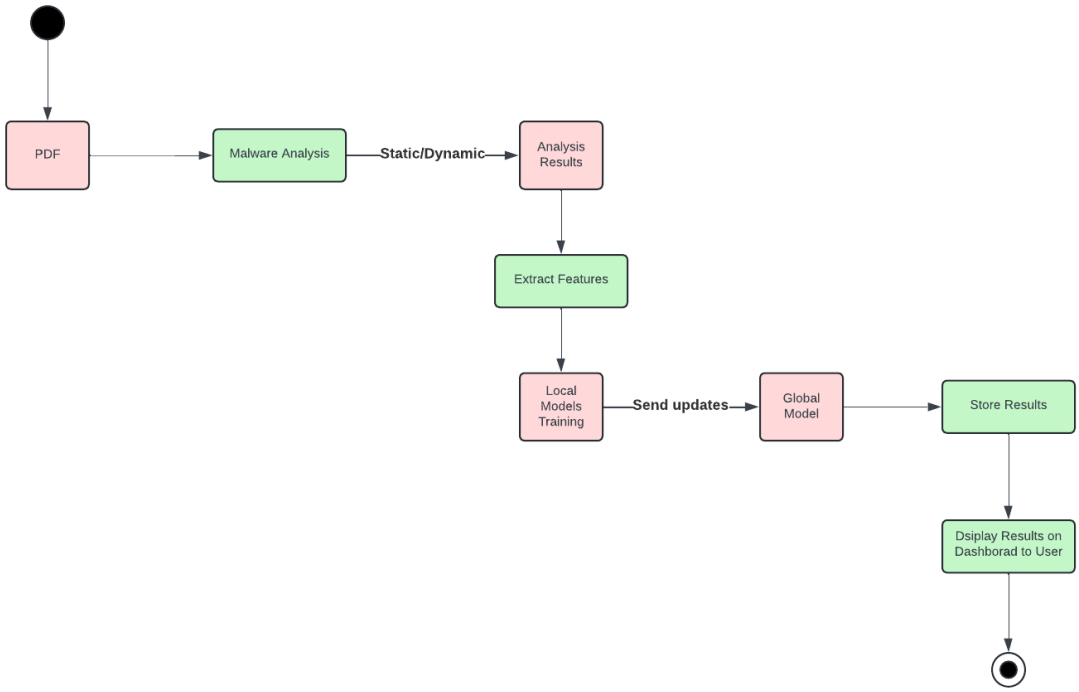

Figure 3.2.5

Sequence Diagram:

The sequence diagram is based on the communications and interactions between the different system components and user.

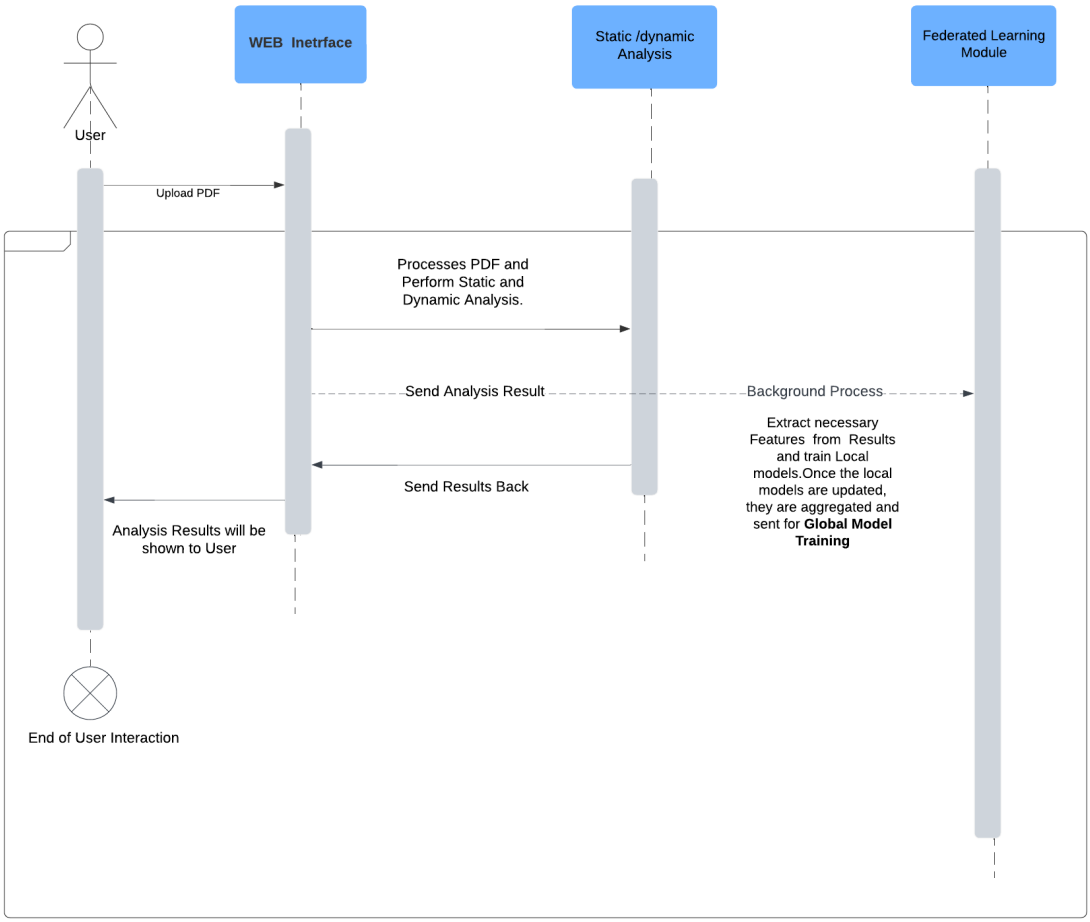

Figure 3.2.6

State Transition Diagram:

A state transition diagram is a graphical representation that shows the different states of a system and the transitions between them. In our malware analysis system, there are various states the system transitions through, such as file upload, malware analysis, and result visualization. Therefore, we have chosen a state transition diagram to represent these processes. The diagram shows how the system moves from the "PDF Upload" state to "Malware Analysis," and depending on success or failure, transitions to either storing results or prompting the user to re-upload. Successful analysis also triggers the training of local and global models through federated learning, enhancing overall system detection capabilities.

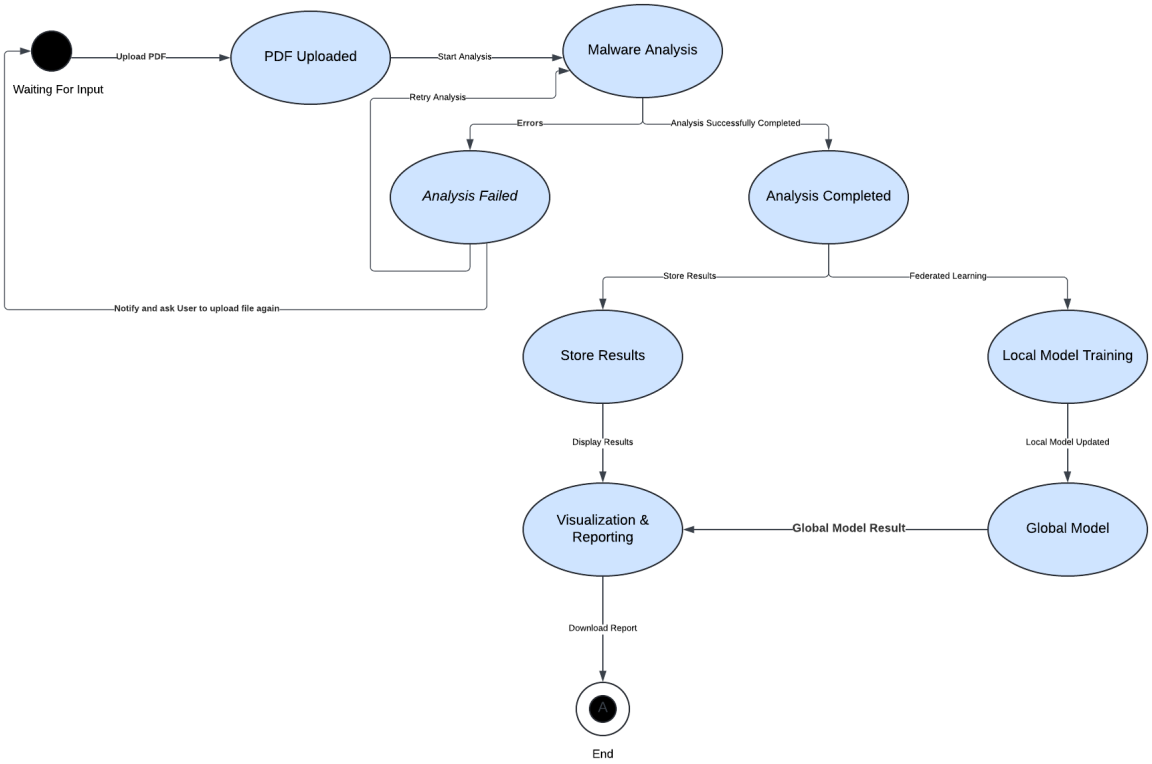

Figure 3.2.7

Data Flow Diagrams (DFDs):

In our malware analysis system, the focus is on the flow of data and the interactions between different components, such as the user, Cuckoo Sandbox API, Elasticsearch, and the admin. Therefore, we have chosen a DFD (Data Flow Diagram) to visualize this aspect of our system. The DFD shows the flow of data in the malware analysis process, where a user uploads a PDF file, the analysis is done via the Cuckoo Sandbox API, and the results are stored in Elasticsearch, with the admin managing system updates and requests.

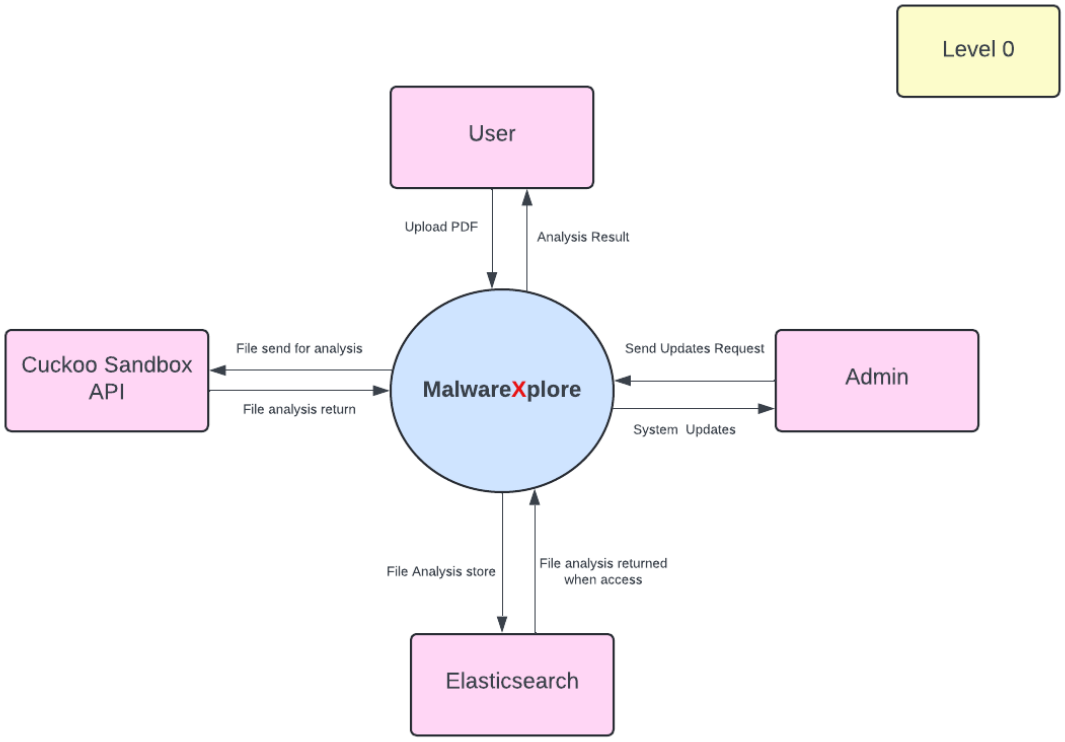

Figure 3.2.8

The Level 1 DFD illustrates the detailed interactions between the user, admin, and various system components like the Cuckoo Sandbox API and Elasticsearch. It shows how a user uploads a PDF for malware analysis, the results are processed, stored, and displayed, while the admin manages system access and updates.

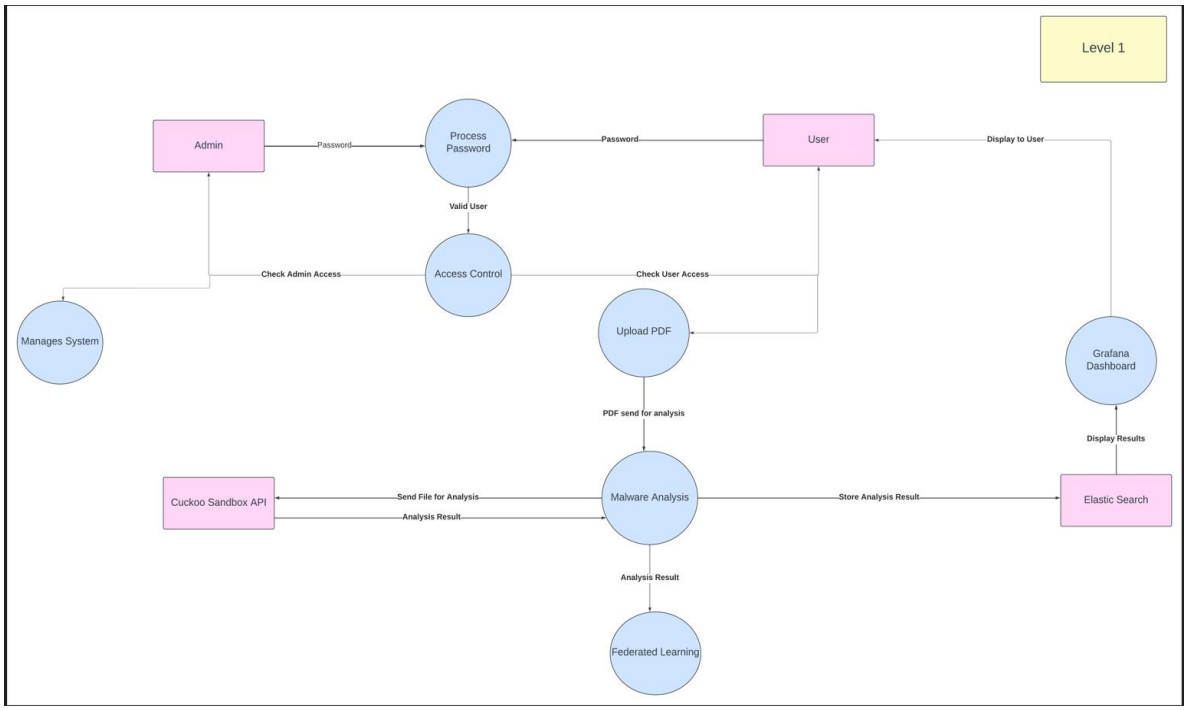

Figure 3.2.9

The **Level 2a DFD** provides a more detailed breakdown of system processes. It focuses on the internal operations, such as **password processing**, **access control**, and how the **admin manages the system**. It also shows interactions like **checking for system updates**, monitoring the **performance of the tool**, and ensuring **system security**. Additionally, there are processes for handling **privacy settings for sharing models** and managing updates related to the **AI model training** and **new PDFs added for analysis**.

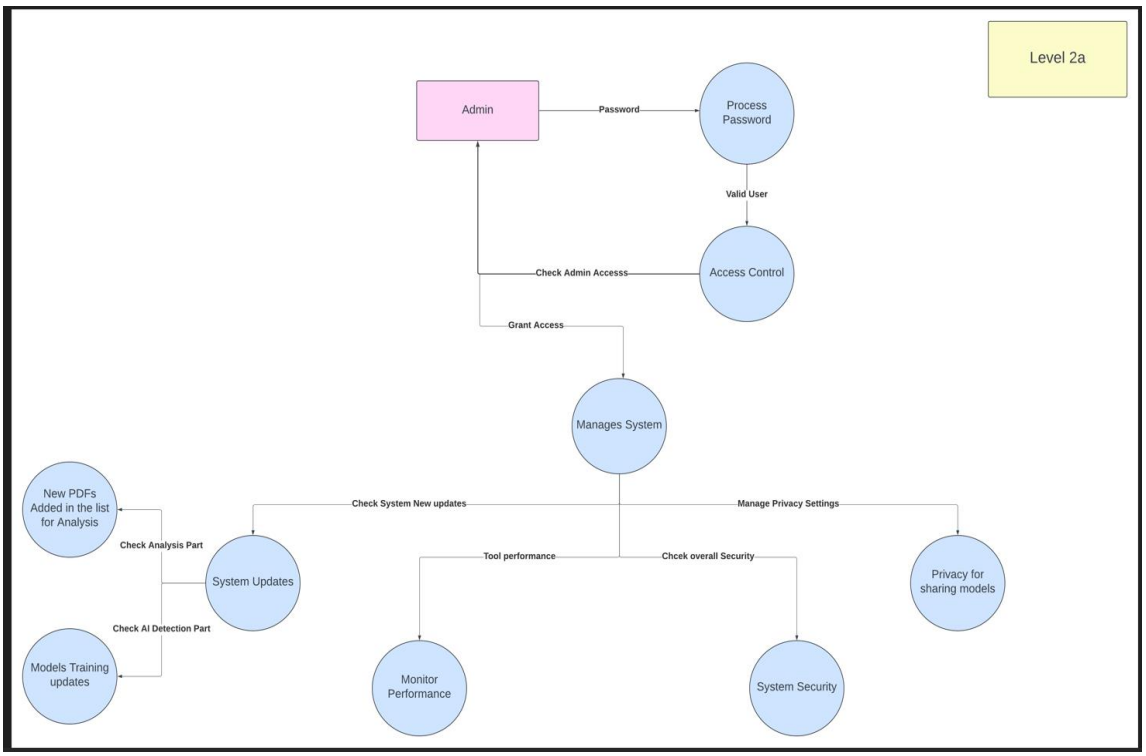

Figure 3.2.10

The **Level 2b DFD** further details the user's interactions within the malware detection system. It shows how the **user uploads a PDF** for analysis, which then goes through both **static** and **dynamic analysis**. The **Cuckoo Sandbox API** is utilized for dynamic analysis, while static analysis is conducted internally. The results from both analyses are sent to the **Elastic Search** component for processing, and then displayed on the **Grafana Dashboard** for the user. The diagram also includes processes like **federated learning**, **local model training**, and **global model training**, which ensure the system continually improves its detection capabilities.

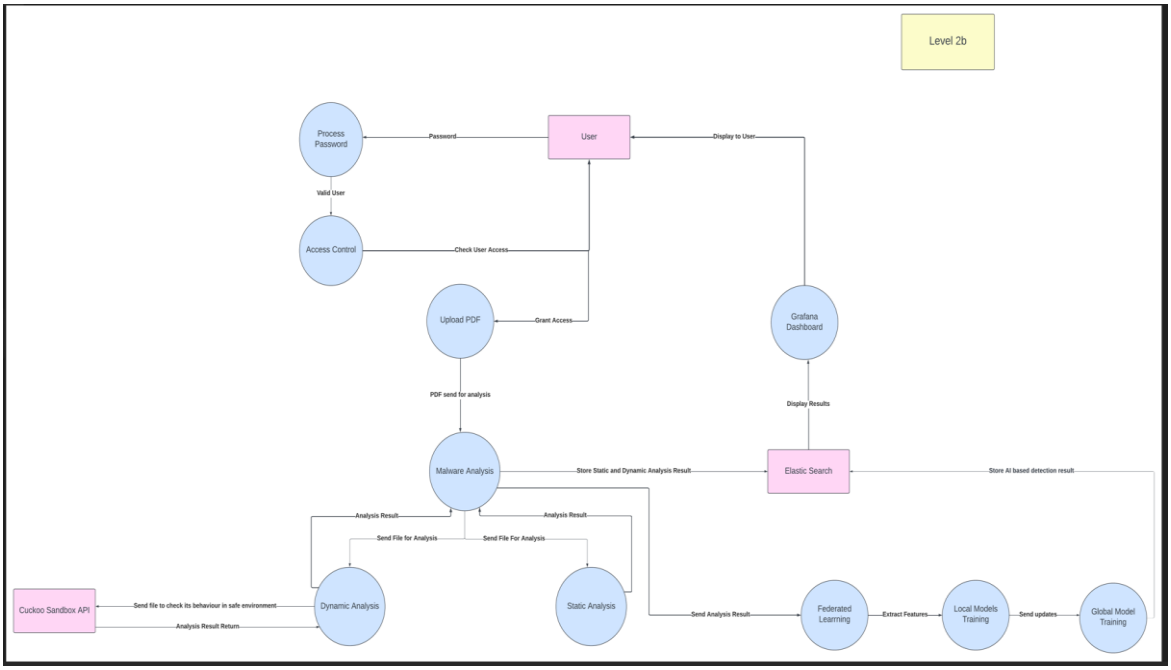

Figure 3.2.11

3.3 Data Design

In our system, the information domain is transformed into data structures by organizing the main entities (USER, PDF, Result, Analysis, and Federated Learning) into tables. Each entity has associated attributes, such as UserID, FileID, and ResultID, which are stored as fields in these tables. The relationships between entities are represented as foreign keys, such as users uploading multiple PDFs.

Data is stored in relational databases, where USER, PDF, Result, Analysis, and Federated Learning are structured as tables. Data processing involves users uploading PDFs, which are analyzed, and the results are then sent to the Federated Learning module for further analysis. The system uses a relational database (e.g., ***MySQL***) and ***Elasticsearch*** as database to manage and organize this information.

ERD:

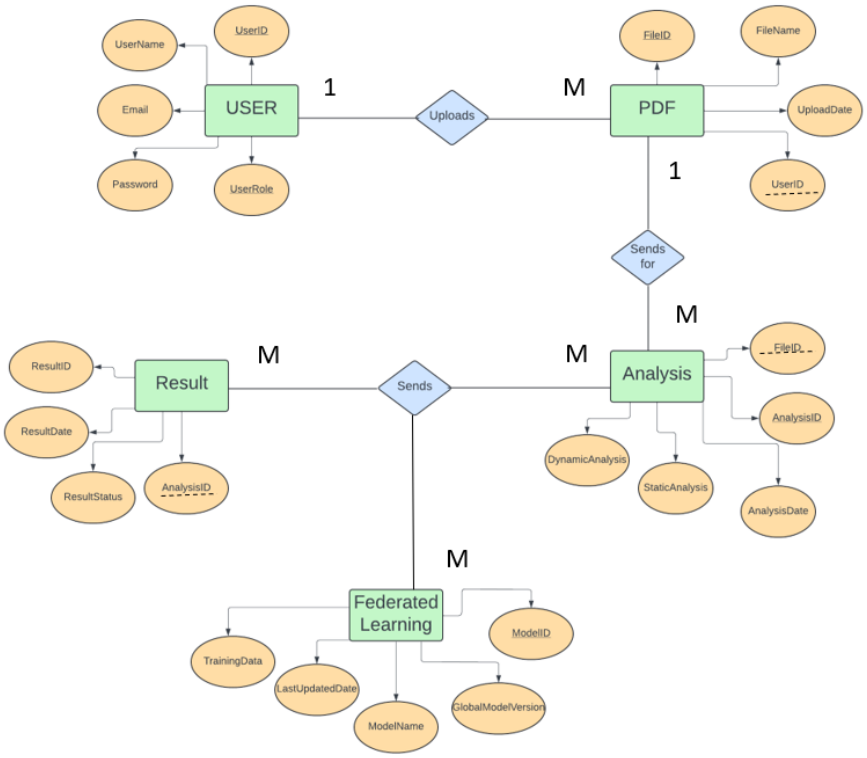

Figure 3.3.1

Bibliography